Università degli Studi di Modena e Reggio Emilia

Facoltà di Ingegneria

CORSO DI

FONDAMENTI DI INFORMATICA
Linguaggio Java: i Thread

Ing. Franco Zambonelli

Lucidi realizzati in collaborazione con
Ing. Enrico Denti - Univ. Bologna

Anno Accademico 1999/2000

PROCESSI

Nei sistemi concorrenti, il termine **processo** denota la *singola attività* eseguita in modo indipendente e separato rispetto alle altre.

- Tipicamente, un programma è eseguito da un processo.
- Il processo è l'istanza in esecuzione di un processo
- Più processi possono eseguire lo stesso programma.

Paragone con Classi e Oggetti: un programma agisce come una classe, il processo rappresenta una instanza in esecuzione del programma.

> In Unix, un nuovo processo viene creato tramite la chiamata di sistema `fork()`, che duplica il processo che la esegue (*processo padre*) creando un identico *processo figlio*.
>
> Il figlio può poi o continuare a eseguire lo stesso codice del padre, o eseguirne un altro tramite la primitiva `exec()`.

Un processo:

- è del tutto autonomo e indipendente dagli altri processi
- ha un prorpio *spazio di indirizzamento riservato* e non condivide memoria con altri processi: significa che anche le variabili globali sono private al processo (duplicazione dell'area di memoria) e i puntatori puntano comunque a cose diverse (rilocazione dei puntatori)
- ha un proprio *stack riservato* (le variabili locale sono pivate al processo).

La comunicazione tra processi distinti non può quindi avvenire in modo "diretto" (per condivisione di variabili), ma deve sfruttare appositi meccanismi di sistema (p.e., file).
I processi servono ad avere attività in esecuzione concorrente MA indipendente, minimizzando le possibilità di interazione, e quindi di disturbo reciproco

THREAD

Spesso si vogliono realizzare programmi in cui non vi è solo una attività in esecuzione, ma in cui diverse attività in concorrenza eseguono (motivi di efficienza dell'esecuzione). In questo caso, vi è può l'esigenza forte di fare interagire le diverse attività in esecuzione.

Un thread (o *processo leggero*):

- è un'attività autonoma che **"vive" *all'interno di un processo*** (e quindi, tipicamente, la stessa istanza in esecuzione di un programma, corrispondente a un processo, ospita al suo interno dei sotto-flussi di esecuzione concorrente, corrispondenti ai thread)
- ha un proprio *stack riservato* (e quindi le sue variabili locali sono private e non condivise da altri thread)
- **..ma non ha uno spazio di indirizzamento riservato**: ***tutti i thread appartenenti allo stesso processo condividono il medesimo spazio di indirizzamento*.**

La comunicazione fra thread può quindi avvenire in modo molto più semplice, tramite lo spazio di memoria condiviso. Quindi, se ci sono variabili globali o riferimenti a aree di memoria od oggetti comuni diversi thread possono interazione.

Occorre però *disciplinare l'accesso* a tale area comune → necessità di opportuni *meccanismi di sincronizzazione.*

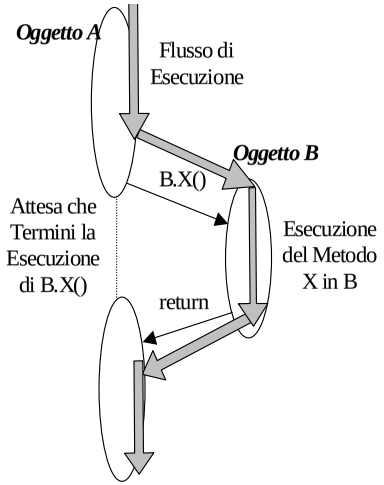
In un liguaggio ad oggetti: ovviamente, il concetto di variabile globale non ha senso. Però più thread possono condividere riferimenti allo stesso oggetto, interagiscono tramite tale oggetto.

THREAD IN JAVA

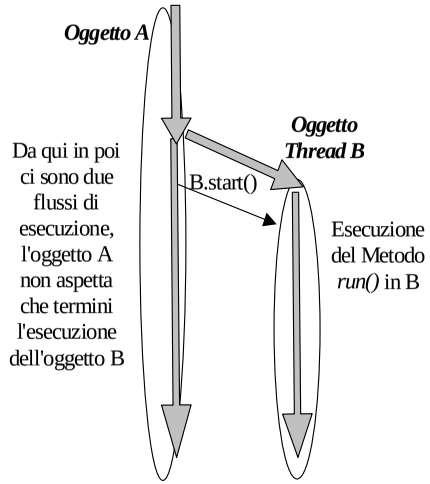
Come si può realizzare il concetto di Thread in Java?

NEL MODO PIU' NATURALE! Sono oggetti particolari ai quali si richiede un servizio (chiamato `start()`) corrispondente al lancio di una attività, di un thread! ***MA***: non si aspetta che il servizio termini, esso procede in concorrenza a chi lo ha richiesto!

Normale Richiesta di Servizio



Richiesta di Servizio start() a un Thread



LA CLASSE `Thread`

La classe Thread è la base per la gestione dei thread in Java.

Essa comprende metodi per attivare, fermare, sospendere, riprendere, attendere thread, per controllarne la priorità, lo scheduling, etc.

Il corpo del thread è costituito dal metodo `run()`

Quindi, per definire una nuova classe di thread si deve:

- definire una propria sottoclasse di Thread che ridefinisca opportunamente il metodo `run()`

Per creare un nuovo oggetto thread si dovrà quindi:

- creare una istanza della propria sottoclasse di `Thread` sopra definita

Per far partire il nuovo thread (cioè l'esecuzione del suo metodo `run()`)basta poi invocare su di esso il metodo `start()`.

ESEMPIO

Creazione di un thread a partire dalla classe `Thread`

La nuova classe di thread…

```
class MyThread1 extends Thread {
  public void run(){
    for(int i=1; i<=10; i++)
      System.out.println(i + " " + i*i);
  }}
```

… e la corrispondente classe di test che lo crea e lo attiva:

```
public class Esempio1 {
  public static void main(String args[]){
    MyThread1 t1 = new MyThread1();
    t1.start();
    // o anche: new MyThread1().start();
  }}
```

NOTE:

- la versione con la variabile `t1` esplicitamente definita è utile qualora su t1 debbano essere invocati altri metodi oltre all'iniziale `start()`
- il nuovo thread continua (in questo caso) la propria esecuzione fino alla sua terminazione naturale; altrimenti, potrebbe essere *espressamente fermato* da un altro thread che invochi su di esso il metodo `stop()`.
- ogni Thread ha sempre associato un nome univoco, corrispondente a una stringa, che o viene assegnato di default dal sistema o viene assegnato dall'utente in fase di inizializzazione! Provare:

```
Thread.currentThread().getName();
```

L'INTERFACCIA `Runnable`

Il metodo run() dei Thread è anche dichiarato nella interfaccia `Runnable`. Quindi, come metodo alternativo per definire dei thread si può:

- definire una propria classe che *implementi l'interfaccia Runnable*, definendo il metodo `run()`.

Questa via è più flessibile, (rispetto all'ereditare dalla classe `Thread`) in quanto consente di definire classi di thread che non siano per forza sottoclassi di `Thread` ma che siano sottoclassi di altri classi generiche.

Per creare e usare un thread bisogna:

- *prima* creare una istanza della propria classe (che implementa l'interfaccia Runnable)
  - *poi* creare una istanza di Thread a partire da tale oggetto.

ESEMPIO

Creazione di un thread tramite l'interfaccia `Runnable`

La nuova classe di thread…

```
class MyBody3 implements Runnable {
 // NON DERIVA DA Thread!
  public void run(){
    for(int i=1; i<=10; i++)
      System.out.println(i + " " + i*i);
  }
}
```

… e la corrispondente classe di test che lo crea e lo attiva:

```
public class Esempio3 {
  public static void main(String args[]){
    MyBody3 b3 = new MyBody3();
    Thread t = new Thread(b3);
    // o anche:
    // Thread t = new Thread(new MyBody3());
    t.start();
  }
}
```

CONTROLLO DEI THREAD

- un thread continua la propria esecuzione o fino alla sua terminazione naturale, o fino a che o fino a che non viene *espressamente fermato* da un altro thread che invochi su di esso il metodo `stop()`
- un thread può anche essere *sospeso* temporaneamente dalla sua esecuzione
  - perché un operazione blocca l'esecuzione, come una operazione di input che implica di attendere dei dati. Il Thread si blocca finchè i dati non sono disponibili.
  - perché invoca la funzione `Thread.sleep();`
  - da un altro thread che invochi su di esso il metodo `suspend()`, per *riprendere* poi la propria esecuzione quando qualcuno invoca su di esso il metodo `resume()`.

La classe di thread opportunamente modificata…

```
class MyThread2 extends Thread {
  public void run(){
    for(int i=1; true; i++) // ciclo infinito
      System.out.println(i + " " + i*i);
  }}
```

… e la corrispondente classe di test che crea e gestisce il thread:

```
public class Esempio2 {
  public static void main(String args[]){
    MyThread2 t2 = new MyThread2();
    t2.start();
    Thread.sleep(1000); // sleeps 1 sec
    t2.suspend();
    Thread.sleep(1500); // sleeps 1,5 sec
    t2.resume();
    Thread.sleep(300);  // sleeps 0,3 sec
    t2.stop();
  }}
```


stop, suspend, resume: PROBLEMI

ATTENZIONE**!** `stop()`, `suspend()` e `resume()` **sono** deprecati **in Java2, in quanto** *non sicuri.*

Questi tre metodi *agiscono in modo brutale* su un thread (che non può "difendersi" in nessun modo), e così facendo **rischiano di creare *situazioni di blocco* (deadlock)**:

- se il thread sospeso / fermato stava facendo un'operazione critica, essa *rimane a metà* e l'applicazione viene a trovarsi in uno stato *inconsistente*
- se il thread sospeso / fermato aveva acquisito una risorsa, essa *rimane bloccata da quel thread* e nessun altro può conquistarla
- se a questo punto un altro thread si pone in attesa di tale risorsa, si crea una situazione di deadlock.

E infatti, se nell'esempio precedente si aggiungono delle `println()` nel main, tutto si pianta alla prima `suspend()`!!

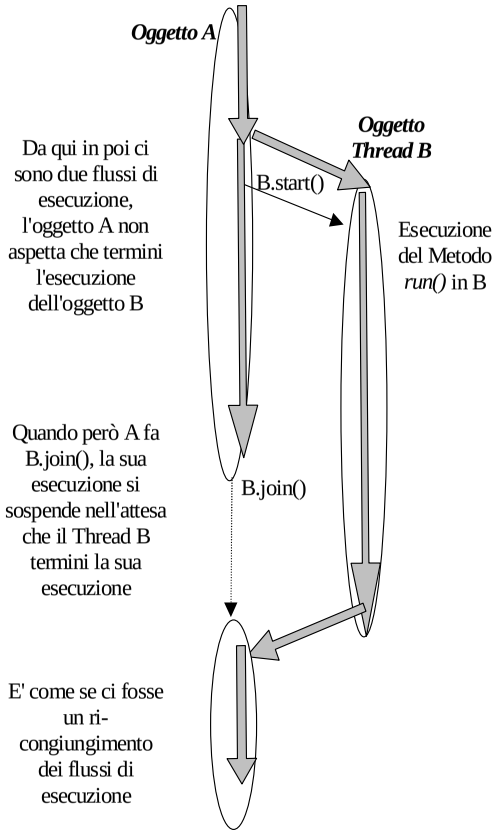
LA SOLUZIONE

La soluzione suggerita da Java 2 consiste nell'adottare un *diverso approccio*:

- *non* agire d'imperio su un thread,
- ma *segnalargli l'opportunità* di sospendersi / terminare,
- **lasciando però al thread il compito di *sospendersi* / *terminare***, in modo che possa farlo in un momento "non critico".
- la funzione ***yeald()*** permette a un thread di cedere l'esecuzione ad un altro!

CONTROLLO DEI THREAD: JOIN

Si può decidere di aspettare che termini l'esecuzione di un thread. Per fare questo, bisogna invocare l'operazione join() sul thread che si intende aspettare



ESEMPIO DI JOIN

Creazione di una serie di thread in sequenza

```
class Esempio4 {
  public static void main(String args[]){
    System.out.println("Main - inizio");
    MyBody4 b4 = new MyBody4();
    while(true) {
      Thread t = new Thread(b4);
      System.out.print("new " +
         (t==null ? "fallita" : "riuscita"));
      System.out.println(" (CTRL-C per finire)");
      t.start();
      try {
        t.join();  // attende che t termini
      }
      catch(InterruptedException e) {}
    }  } }
```

```
class MyBody4 implements Runnable {
  public void run(){
      System.out.println("Qui thread " +
        Thread.currentThread().getName() );
  }
}
```

NOTE:

- il main è un ciclo infinito che crea continuamente nuovi thread (fino a che non viene chiuso con CTRL-C)
- prima di creare un nuovo thread, il main attende, tramite `join()`, che termini quello precedentemente in esecuzione
- l'attesa di `join()` potrebbe essere interrotta da un'eccezione → necessità di try/catch

UNA VARIANTE:

Creazione di una serie di thread *con nome* in sequenza

```
class Esempio4bis {
  public static void main(String args[]){
    System.out.println("Main - inizio");
    MyBody4 b4 = new MyBody4();
    int i=1;
    while(true) {
      Thread t = new Thread(b4, "Enrico" + i++);
      System.out.print("new " +
         (t==null ? "fallita" : "riuscita"));
      System.out.println(" (CTRL-C per finire)");
      t.start();
      try {
        t.join();  // attende che t termini
      }
      catch(InterruptedException e) {}
    }  }}
```

```
class MyBody4 implements Runnable {
  public void run(){
      System.out.println("Qui thread " +
        Thread.currentThread().getName() );
  }}
```

NOTA:

- la classe `Thread` ha sia un costruttore con un solo argomento (che si aspetta un oggetto che implementi `Runnable`) *sia uno con due argomenti*, che all'oggetto `Runnable` affianca un oggetto `String` che si assume essere *il nome del thread*.

SCHEDULING

In un sistema monoprocessore i thread devono condividere l'esecuzione su un unico processore. A un certo istante, ci sarà un solo thread realmente in esecuzione, e altri in attesa di acquisire il controllo del processore.

Quando viene rilasciato il processore?

- quando si involcano dei metodi di controllo, come `sospend(), resume, stop(), che` permettono di gestire l'uso del processore, e fare in modo che un thread, bloccandosi o sospendendosi, ceda il processore.
- quando un processo invoca una operazione che implica attesa di qualche evento, come un'operazione di input, durante la quale attesa non ha bisogno del processore;

Nei casi sopra, il controllo del processore viene ovviamente ceduto a un altro thread. Altrimenti, se i thread non vengono esplicitamente bloccati o se non fanno operazioni che implicano attesa:

- *scheduling nonpreemptive* se il controllo del microprocessore non viene mai forzatamente tolto al thread corrente
- *scheduling preemptive* in condivisione di tempo (*time sharing*) se i thread possono eseguire ininterrottamente solo per un certo intervallo di tempo, dopo di che sono forzati a cedere il microprocessore algi altri thread in attesa..

Java non precisa quale tipo di multitasking debba essere adottato dalla macchina virtuale! ***Dipende dal Sistema Operativo!***

Però, non è difficile verificarlo con un piccolo test.

LA POLITICA DELLO SCHEDULER

ESEMPIO

```
class Esempio5 {
  public static void main(String args[]){
    System.out.println("Main - inizio");
    MyBody5 b5 = new MyBody5();
    new Thread(b5, "cip ").start();
    new Thread(b5, "ciop").start();
    // NB: se è non-preemptive, va solo cip
    // se è preemptive, vanno entrambi
  }
}
```

```
class MyBody5 implements Runnable {
  public void run(){
    while (true)
      System.out.println(
        Thread.currentThread().getName() );
  }
}
```

NOTA:

- `cip` e `ciop` sono due thread che eseguono lo stesso codice `b5`, che è un ciclo infinito: quindi, in presenza di scheduling non preemptive andrà sempre e solo `cip`. Se vanno entrambi, lo scheduling è certamente preemptive (*caso JDK Windows*).

COROUTINING

Indipendentemente dalla politica di scheduling dei adotata dalla macchina virtuale, è facile in Java costruire due thread che si cedono reciprocamente il controllo (schema cosiddetto di co-routining)

ESEMPIO 6 - Esempio di coroutining fra due thread

```
class Esempio6 {
  public static void main(String args[]){
    System.out.println("Main - inizio");
    MyBody6 b6 = new MyBody6();
    new Thread(b6, "cip ").start();
    new Thread(b6, "ciop").start();
  }}
```

```
class MyBody6 implements Runnable {
  public void run(){
    while (true) {
      System.out.println(
           Thread.currentThread().getName());
      Thread.yield(); // cede l'esecuzione ad un
                      // altro thread
    }  } }
```

Dopo avere scritto il nome, un trehad cede il controllo all'altro thread, rimettendosi in coda in attesa di poter ri-acquisire l'esecuzione.

NOTA:L'output può non essere un rigoroso alternarsi `cip` / `ciop`, perché in mezzo può inserirsi lo scheduler (preemptive).

SINCRONIZZAZIONE di THREADS

Quando due o più thread eseguono concorrentemente, è in generale impossibile prevedere l'ordine in cui le loro istruzioni verranno eseguite.

Problemi nel caso in cui i thread invocano metodi sullo stesso oggetto di cui condividono il riferimento. ***SONO POSSIBILI INCONSISTENZE!***

Esempio:
Oggetto `conto_corrente`
con metodo `versamento(importo)`

```
public void versamento(int importo)
{ int nuovo_totale; //variabile locale

nuovo_totale = totale_conto + importo - tasse;
//totale_conto è una variabile dell'oggetto
//e indica i soldi totali sul conto
//l'istruzione sopra calcola in nuovo totale del
// conto corrente mettendo il risultato nella
//variabile locale

totale_conto  = nuovo_totale;
// metto il totale calcolato nella variabile
//dell'oggetto che memorizza il conto totale
}
```

Supponiamo che due thread abbiano entrambi un riferimento all'oggetto invochino separatamente il metodo versamento per fare ognuno un versamento sul conto….

ESEMPIO DI INCONSISTENZA

Supponiamo che l'oggetto `conto_corrente` abbia nella sua variabile `totale_conto` il valore **2000**. Ci si aspetta che se qualcuno deposita **500** e qualcun altro deposita **1000**, e supponendo che le tasse per ogni versamento siano **100**, alla fine la variabile totale_conto valga **3300**.

Supponiamo che questi due depositi vengano fatti in concorrenza da due thread diversi e che durante l'esecuzione i thread si alternino sul processore, come segue

Thread A	Thread B
conto_corrente.versamento(1000) // il thread invoca il metodo e il flusso di esecuzione fa a eseguire le istruzioni di tale metodo	
	conto_corrente.versamento(500) // il thread invoca il metodo e il flusso di esecuzione fa a eseguire le istruzioni di tale metodo
nuovo_totale= totale_conto+importo-tasse; //nuovo_totale = 2000+1000-100	
	nuovo_totale= totale_conto+importo-tasse; //nuovo_totale = 2000+500-100
totale_conto =nuovo_totale; //totale_conto vale 2900	
	totale_conto = nuovo_totale; //totale_conto vale 2400

Alla fine,* `totale_conto` *vale 2400!!!!

SEZIONI CRITICHE

Le sezioni di codice in cui si accede a risorse e oggetti comuni (condivise) sono *critiche*: per evitare inconsistenze, è necessario assicurare che, prima di entrarvi, i thread *si sincronizzino*, in modo da garantire che solo un thread per volta possa eseguire la sezione critica. Nell'esempio precedente, bisogna evitare che due thread eseguano in concorrenza in metodo `versamento`.

In Java si può garantire l'accesso in mutua esclusione a una istanza, proteggendo **il metodo** o **la sezione di codice critica** tramite la keyword **`synchronized`:**

- metodo sincronizzato:

  ```
  public synchronized void setValue() {...}
  ```

  Solo un thread alla volta può eseguire questo metodo sullo stesso oggetto.

- sezione di codice sincronizzata:

  ```
  synchronized(object) {...}
  ```

  Solo un thread alla volta può eseguire la parte di codice protetta sull'oggetto `object` (che può essere `this`).

Nell'esempio precedente:

- il metodo **`versamento`** deve essere **`synchronized`**
- mentre il flusso di esecuzione di un thread esegue tale metodo, è garantito che nessun'altro thread potrà eseguirlo
- se *Thread A* sta già esegunedo il metodo, quando *Thread B* tenta di eseguirlo viene sospeso, in attesa che *Thread A* termini
- quando *Thread A* termina, *Thread B* riprende l'esecuzione a può eseguire il metodo versamento.

NESSUNA INCONSISTENZA!!!!!

ESEMPIO - Uno stack sincronizzato

```
class EmptyStackException extends Exception {
  EmptyStackException() {}
  EmptyStackException(String s) { super(s); }
}
```

```
class Stack {
  Object val[];
  int sp = 0;

  Stack(int max) { val = new Object[max]; }

  public synchronized void push(Object e) {
    val[sp++]=e;
  }

  public synchronized Object pop()
        throws EmptyStackException {
    if (sp>0) return val[--sp];
    else throw new EmptyStackException();
  }

  public boolean isEmpty() { return (sp==0); }

  public void print() {
    System.out.print("Stack content: [");
    for(int i=0; i<sp-1; i++)
        System.out.print(val[i] + ", ");
    if (sp>0) System.out.print(val[sp-1]);
    System.out.print("]");
  }
}
```

ESEMPIO - Uso dello stack

Questo codice riunisce in sé *tre attività diverse*: il pusher (che inserisce nello stack dato 20 oggetti Integer), il popper (che li estrae), e il printer (che visualizza lo stato dello stack).

Ogni istanza di `MyBody9` si comporterà quindi o come pusher, o come popper, o come printer, *a seconda del nome che le viene passato all'atto della costruzione*.

```
class MyBody9 implements Runnable {
  String chiSonoIo;
  static Stack s = new Stack(100); // condiviso
  MyBody9(String name){ chiSonoIo = name; }
  public void run(){
    if (chiSonoIo.equals("pusher")) {
      for(int i=0; i<20; i++) {
        System.out.println(
            Thread.currentThread().getName()
            + " pushing " + i);
        s.push( new Integer(i) );
      } } else
    if (chiSonoIo.equals("popper")) {
      try { Thread.sleep(500); }
      catch (InterruptedException ee) {}
      try {
         while(!(s.isEmpty()))
          System.out.println(
            Thread.currentThread().getName()
              + " popping " + s.pop());
      } catch (EmptyStackException e) {
          System.out.println(
            Thread.currentThread().getName()
            + " tried to pop an empty stack"); }
    } else /* sono il printer */
      s.print();
}}
```


ESEMPIO - Uso dello stack

```
public class Esempio9 {
  public static void main(String args[]){
    // creo tre istanze configurate diversamente
    MyBody9 b1 = new MyBody9("pusher");
    MyBody9 b2 = new MyBody9("popper");
    MyBody9 b3 = new MyBody9("printer");
    // creo tre thread, uno per task da eseguire
    Thread t1 = new Thread(b1, "Produttore");
    Thread t2 = new Thread(b2, "Consumatore");
    Thread t3 = new Thread(b3, "Visualizzatore");
    // ora attivo Produttore e Consumatore…
    t2.start();
    t1.start();
    // …e aspetto che finiscano entrambi
    try { t1.join(); }
    catch(InterruptedException e1) {}
    try { t2.join(); }
    catch(InterruptedException e2) {}
    // alla fine attivo il Visualizzatore
    t3.start();
  }
}
```

NOTA:

- a rigore, esiste ancora un rischio di inconsistenza *sugli oggetti memorizzati nello stack*, in quanto si memorizzano *riferimenti*: altri thread potrebbero fare riferimento agli stessi oggetti, e modificarli *mentre sono memorizzati* nello stack
- sarebbe più sicuro archiviare nello stack delle copie (*cloni*) degli oggetti anziché mantenere dei riferimenti